

My Notes on Inference Engineering

Lecture 1 & 2

Just writing this down in my own words so it actually sticks – how LLMs generate text, how much memory they eat up, and how to pick hardware/software for running them.

What I'm covering here

- ✓ Why inference feels slow, how to guess memory needs, what the whole stack looks like
- ✓ Comparing two GPUs I keep hearing about
- ✓ Two serving tools that weren't in class
- ✓ How to actually read `nvidia-smi`

Part 1: The Basics

1. Why is inference "memory-bound"? (i.e. why is it slow?)

Okay so the main thing to get here: when a model generates text, it's not slow because the GPU can't do the math fast enough. It's slow because the GPU is stuck *waiting* to pull the model's weights out of memory. Compute isn't the bottleneck – memory access is.

The way I'm picturing it: imagine a chef, and the ingredients are sitting in a fridge. To make one dish (= generate one token), the chef has to go grab every single ingredient, even if they only need a tiny pinch of it. If the fridge is slow or far away, the chef spends way more time walking back and forth than actually cooking. That's basically what's happening:

- To spit out just ONE new word, the GPU has to read through the *entire* model's weights.
- Reading all that data takes way longer than the actual math needed once it's loaded in.
- So the math part is fast, the "fetching data from memory" part is the slow part.

There's an actual term for this: **arithmetic intensity**.

$$\text{Arithmetic Intensity} = \frac{\text{how much math you do}}{\text{how much data you had to read}}$$

Every GPU has a max speed for doing math (FLOPs/s) and a max speed for reading memory (bytes/s). Wherever these two lines cross is called the **ridge point**. If your task's arithmetic intensity is below that point, you're **memory-bound** – basically limited by how fast you can fetch data, not how fast you can compute. And generating text one word at a time? Almost always below that line.

Why I should actually care about this:

- Buying a GPU with crazy high TFLOPs doesn't help much if it's already bottlenecked on memory reads.

- What actually helps: (1) making the model smaller so there's less to read (this is quantization), or (2) serving lots of requests at the same time so each memory trip is used more efficiently (this is batching).
- So basically – when picking/renting a GPU for running models, **memory bandwidth (GB/s)** matters just as much if not more than raw TFLOPs. Good to remember before I go buy the "fastest" GPU on paper.

2. How do I figure out how much memory a model needs?

Simple version: a model is just a giant list of numbers (parameters), so the memory it needs = how many numbers there are $\times$ how many bytes each number takes.

How the numbers are actually stored: An LLM is made of layers (attention layers, feed-forward layers, etc.), and each layer is full of matrices of numbers – these are the parameters/weights. They get saved to disk (usually as safetensors files), and each number takes up a fixed number of bytes depending on the precision used:

Precision	Bytes/number	When it's used
FP32 (full)	4 bytes	Mostly training
FP16 / BF16 (half)	2 bytes	The normal choice for serving
INT8 / FP8	1 byte	Common "quantized" option
INT4 (4-bit)	0.5 bytes	For squeezing onto small GPUs

The formula I need to remember:

$$\text{Memory needed} = \text{number of parameters} \times \text{bytes per parameter}$$

Quick example so it clicks: A 7B model in FP16 (2 bytes each):

$$7,000,000,000 \times 2 \text{ bytes} = 14,000,000,000 \text{ bytes} \approx 14 \text{ GB}$$

But wait – that's not the whole picture. When the model is actually running, it also needs room for:

- **KV cache** – kind of like a scratchpad that remembers stuff about every word processed so far, so it doesn't redo work. Grows the longer the chat gets, and the more people using it at once.
- **Activations** – short-lived memory used while it's computing.
- **Overhead** – just the normal extra memory the software needs.

Rule of thumb to remember: take the weight memory, add ~20% on top for overhead, and then budget the KV cache separately depending on how many users / how much context I expect.

3. What does the whole "inference stack" actually look like?

The big idea here: serving a model isn't just "run the model." It's a bunch of layers stacked on top of each other – kind of like how a restaurant needs more than just a chef; it needs a kitchen, a menu, waiters, a manager, etc.

1. **Hardware layer** – the actual GPUs (memory size, memory speed, how they’re connected to each other).
Why it matters: this is usually the biggest cost. Pick wrong and you either can’t run the model at all, or you’re way overpaying.
2. **Model/Weights layer** – how the model is stored/compressed (full precision vs quantized).
Why it matters: shrinking the model can let you use 2–4x cheaper hardware for almost the same quality. Straight up saves money.
3. **Inference engine layer** – the software that runs the model efficiently (vLLM, TensorRT-LLM, TGI, etc). Handles the batching and memory tricks.
Why it matters: a good engine = 5–10x more users on the exact same hardware vs. a naive setup. This is where most of the “free” speed comes from, apparently.
4. **Serving/orchestration layer** – takes in requests, spreads them across GPUs, scales up/down when traffic changes.
Why it matters: keeps things online and fast when traffic spikes.
5. **Application layer** – APIs, safety filters, dev tools built on top.
Why it matters: decides how fast teams can ship features, and it’s where safety rules get enforced.
6. **Monitoring & cost layer** – dashboards/logs tracking speed, errors, and \$\$\$.
Why it matters: without this you have zero idea if you’re wasting money or about to have an outage. GPUs are expensive so this stuff matters.

If even one layer here is weak (bad engine, wrong GPU, whatever), it can bottleneck the WHOLE system no matter how good the model itself is. That’s basically why “inference engineering” is its own job/field.

Part 2: Hardware & Software

4. Comparing two GPUs – A100 vs RTX 4090

Picked these two because they’re so different: the **A100 80GB** (big expensive datacenter GPU) vs the **RTX 4090** (cheaper, the one regular people buy).

Spec	A100 80GB (SXM)	RTX 4090
VRAM (memory size)	80 GB	24 GB
Memory bandwidth (speed)	≈2.0 TB/s	≈1.0 TB/s
Cloud rental cost	≈\$1.05–1.49/hr	≈\$0.35–0.45/hr

Quick way to guess the biggest model a GPU can fit:

$$\text{Max params (billions)} \approx \frac{\text{Usable VRAM (GB)}}{\text{Bytes per parameter}}$$

(Using ~85–90% of total VRAM as “usable” since some is always needed for overhead.)

Biggest model that fits	A100 80GB	RTX 4090
At 16-bit	≈34–36B params	≈10–11B params
At 4-bit	≈136–144B params	≈40–42B params

So basically: the A100 can run a big 70B model once it's shrunk to 4-bit, and the much cheaper 4090 is good for smaller 7–8B models at full quality, or a shrunk-down 30B at 4-bit. Note: this is only the weights – still need to leave room for the KV cache from before.

5. Two serving tools we didn't go over in class

(a) Hugging Face TGI

Basically a ready-made server from Hugging Face – deploy almost any model from their site with one command. It auto-handles batching, streams the text back word by word, and is already tuned to run decently fast.

Good for: people already using Hugging Face's stuff who just want something simple that's well documented and works on more than just NVIDIA (runs on AMD too).

Pick this over vLLM/SGLang/TensorRT-LLM when: setup ease + Hugging Face integration matter more than raw top speed.

Side note: apparently Hugging Face put TGI into "maintenance mode" now and tells people to use vLLM or SGLang instead for new stuff. So it's more of a "solid old option" than the first pick today.

(b) LMDeploy

A serving toolkit made by the InternLM team. Known for being really fast, especially once the model is quantized (shrunk). Its engine (TurboMind) is written in C++ not Python, which cuts a lot of the wasted overhead.

Good for: squeezing max speed out of NVIDIA GPUs, especially for 4-bit quantized models – can be up to 2.4x faster than full precision.

Pick this over vLLM/SGLang/TensorRT-LLM when: my model is supported well by TurboMind and raw NVIDIA speed is priority #1. Not great if I need support for lots of different model types or non-NVIDIA hardware – it's pretty narrow/specific.

Part 3: Coding Bit

6. Reading nvidia-smi output

Basically `nvidia-smi` = a dashboard for the GPU. Shows what's installed, how full the memory is, temperature, and what's currently using the GPU.

Here's an example output:

```
NVIDIA-SMI 550.90  Driver: 550.90.07  CUDA Version: 12.4
GPU: 0  A100-SXM4-80GB  Bus-Id: 00000000:00:04.0
Fan: 30%  Temp: 45C  Power: 68W / 400W
```

```
Memory-Usage: 21504MiB / 81920MiB GPU-Util: 37%  
Processes:  
PID 12345 python (vllm serve ...) Memory: 20800MiB
```

Breaking down the 7 things this tells me:

1. **Driver & CUDA version** – the "translator software" between the GPU and stuff like PyTorch. If it's too old, a lot of tools just refuse to run.
2. **GPU name** – confirms exactly which GPU I'm on (e.g. A100-SXM4-80GB), so I know my memory/speed limits.
3. **Memory-Usage (21504MiB / 81920MiB)** – how full the memory is. #1 thing to watch – if it gets too close to full, I get an out-of-memory crash.
4. **GPU-Util (37%)** – how much of the time the GPU was actually crunching numbers vs just waiting. Low number during heavy use = it's stuck waiting on memory, not on math (ties back to point 1 above!).
5. **Power usage (68W / 400W)** – power draw close to max = GPU is actually working hard. Power draw way below max = probably just sitting idle waiting for data.
6. **Temp & fan speed (45C, 30%)** – if the GPU gets too hot it auto-throttles (slows itself down) to protect itself – and it does this quietly, no error message, so worth keeping an eye on.
7. **Processes list** – shows exactly which programs are using the GPU and how much memory each one is holding onto. Useful for catching a leftover "zombie" process still hogging memory even when nothing looks active.

Good warm-up before next class, which apparently goes deeper into how GPU commands actually get scheduled and run under the hood.